

VISION PAR ORDINATEUR

SCIENCE, TECHNIQUES ET APPLICATIONS

Pr. Adil CHERGUI
Département Informatique

*Master Big Data et
Internet des Objets
Saison - 2023/2024*

TP 4 : Les fonctions OpenCV de transformation d'image

Le traitement d'image est une technique permettant d'appliquer certains algorithmes mathématiques et d'utiliser diverses fonctions et techniques pour manipuler des images numériques afin d'obtenir les résultats souhaités avec des effets améliorés, tels que l'amélioration de leur qualité ou l'extraction d'informations utiles.

OpenCV fournit un certain nombre de méthodes de traitement d'image, ce TP présentera certaines méthodes couramment et largement utilisées pour le traitement d'image.

Ce TP commence par expliquer comment redimensionner, recadrer et faire pivoter les images dans la section 1.

Ce TP présentera également les fonctionnalités orientées objet de Python, nous créerons des classes pour inclure les fonctions et écrirons des codes pour instancier les classes et appeler ces fonctions.

La section 2 expliquera comment ajuster la luminosité et le contraste, et la section 3 pour ajuster la teinte, la saturation et la valeur des images.

En ajustant ces valeurs, la couleur et la luminosité des images peuvent être modifiées en conséquence, c'est une partie très importante du traitement de l'image.

La section 4 présentera la superposition d'images qui consiste à mélanger deux images sur la base d'un algorithme. Et la section 5 présentera l'opération au niveau du bit et le mélange ou le mélange de deux images basées sur l'opération au niveau du bit, ce qui peut obtenir un effet différent.

La section 6 décrira la déformation d'image, une méthode pour transformer une image asymétrique et déformée en une image redressée.

La section 7 présentera le flou d'image, c'est une technique pour rendre l'image lisse en supprimant les bruits de l'image. Il existe de nombreuses façons différentes de le faire, cette section se concentrera sur deux types de méthodes de flou d'image, le flou gaussien et le flou médian.

Enfin, la section 8 présentera l'histogramme qui nous fournit des informations importantes sur les distributions de couleurs des images.

1. Redimensionner, recadrer et faire pivoter une image

Python est un excellent langage pour prendre en charge la programmation orientée objet, cette section commencera à utiliser des techniques orientées objet pour écrire les codes. Nous allons créer des classes pour inclure des propriétés et des fonctions, puis instancier les classes et appeler leurs fonctions.

Créez maintenant une classe appelée **TraitementImage** et définissez trois fonctions à l'intérieur, *redimensionnement()*, *rotation()* et *crop()*. Lorsqu'une classe est créée, elle a toujours une fonction **__init__()** intégrée, qui est toujours exécutée lorsque la classe est instanciée. Il est utilisé pour attribuer des valeurs initiales aux propriétés ou à d'autres opérations nécessaires lors de la création de l'objet. Ici, nous définissons les propriétés *nom_fenetre* et *nom_image* dans la fonction **__init__()**.

```
1 class TraitementImage(object):
2     def __init__(self,nom_fenetre, nom_image):
3         self.nom_fenetre=nom_fenetre
4         self.nom_image=nom_image
5         self.image=cv2.imread(self.nom_image)
6     def affiche(self,titre=None,image=None):
7         if image is None:
8             image=self.image
9         if titre is None:
10             titre=self.nom_fenetre
11         cv2.imshow(titre,image)
```

Explication :

Ligne 1	Définir la classe <i>TraitementImage</i>
Ligne 2	La fonction <i>__init__()</i> intégrée, elle est toujours exécutée lorsque la classe est instanciée. Les paramètres doivent être transmis lorsque la classe est instanciée.
Lignes 3 à 5	Définissez les propriétés de classe, <i>nom_fenetre</i> et <i>nom_image</i> , puis chargez l'image à l'aide de <i>cv2.imread()</i> .
Lignes 6 à 11	Définissez la fonction <i>affiche()</i> , qui affichera l'image en utilisant <i>cv2.imshow()</i> . Lorsque les paramètres spécifient l'image ou le nom de la fenêtre, affichez cette image avec le nom de la fenêtre, sinon utilisez l'image et le nom de la fenêtre spécifiés dans la fonction <i>__init__()</i> lorsque la classe est initialisée.

Pour continuer, ajoutez trois fonctions, *redimensionnement()*, *crop()* et *rotation()*, à la classe :

```
11 def redimensionnement(self,pourcentage,image=None):
12     if image is None:
13         image=self.image
14     largeur=int(image.shape[1]*pourcentage/100)
```

```

15     hauteur=int(image.shape[0]*pourcentage/100)
16     image_redimension=cv2.resize(image,(largeur,hauteur))
17     return image_redimension
18 def crop(self,premier_pt,second_pt,image=None):
19     if image is None:
20         image=self.image
21         #Le point haut-gauche
22         x_hg,y_hg=premier_pt
23         #Le point bas-droit
24         x_bd,y_bd=second_pt
25         #échangée Les x si ils sont opposés
26         if x_bd<x_hg:
27             x_bd,x_hg=x_hg,x_bd
28         #échangée Les y si ils sont opposés
29         if y_bd<y_hg:
30             y_bd,y_hg=y_hg,y_bd
31         image_cropper=image[y_hg:y_bd,x_hg:x_bd]
32         return image_cropper
33 def rotation(self,angle,image=None,echelle=1.0):
34     if image is None:
35         image=self.image
36         (h,l)=image.shape[:2]
37         centre =(l/2,h/2)
38         mat_rot=cv2.getRotationMatrix2D(centre,angle,echelle)
39         image_rotation=cv2.warpAffine(image,mat_rot,(l,h))
40     return image_rotation

```

Explication :

Ligne 11	Définissez la fonction <i>redimension()</i> , spécifiez le pourcentage comme paramètre et éventuellement l'image comme paramètre.
Lignes 12 - 13	Si l'image n'est pas spécifiée dans les paramètres, utilisez l'image de la propriété de classe.
Lignes 14 à 17	Calculez la largeur et la hauteur redimensionnées en fonction du pourcentage. Appelez ensuite la fonction <i>cv2.resize()</i> pour redimensionner l'image.
Lignes 18 à 32	Définissez la fonction <i>crop()</i> , similaire à ce que nous avons fait dans la TP3.
Lignes 33 – 40	Définissez la fonction <i>rotation()</i> , passez l'angle comme paramètre et éventuellement l'image et l'échelle. Utilisez les fonctions <i>cv2.getRotationMatrix2D()</i> et <i>cv2.warpAffine()</i> pour effectuer la rotation.

La définition de la classe pour *TraitementImage* est terminée pour l'instant, d'autres fonctions seront ajoutées ultérieurement.

La classe peut désormais effectuer les opérations de redimensionnement, de recadrage et de rotation.

Dans les codes sources de *RedimentionCropRotation.py*, la classe est importée dans la ligne 2, c'est une manière typique d'inclure un autre fichier Python pour référence.

```

1 import cv2
2 import fonctions.TraitementImage as ti
3
4 if __name__ == "__main__":
5     # Créer un objet de la classe TraitementImage
6     ti = ti.TraitementImage("Originale", 'ressources/image1.jpg')
7
8     # affiche l'image originale
9     ti.affiche(image=ti.redimention(80))
10
11     # Redimentionner l'image Originale et l'afficher
12     image_redim = ti.redimention(50)
13     ti.affiche("Redimentionner -- 50%", image_redim)
14
15     # rotation de l'image redimentionner et l'afficher
16     imge_rotation = ti.rotation(45, image_redim)
17     ti.affiche("Rotourner -- 45 degree", imge_rotation)
18
19     # Cropper l'image original et l'afficher
20     imge_croppe = ti.crop((100, 100), (530, 400))
21     ti.affiche("Cropper", imge_croppe)
22
23     cv2.waitKey(0)
24     cv2.destroyAllWindows()

```

Explication :

Ligne 2	Importez le fichier de classe que nous avons défini précédemment, car il se trouve dans le dossier fonctions, <i>fonctions.nom_fichier</i> est utilisé pour localiser le fichier.
Ligne 4	L'entrée principale, lorsque le code est exécuté, cela commence à partir d'ici.
Ligne 6	Instancier la classe <i>TraitementImage</i> avec les paramètres, le nom de la fenêtre et le chemin d'une image, qui sont définis comme propriétés de cette classe.
Ligne 9	Afficher l'image par défaut et le nom de la fenêtre par défaut, qui sont spécifiés lors de l'instanciation de la classe.
Lignes 12 – 13	Invocuez la fonction <i>redimention()</i> pour redimensionner l'image et l'afficher.
Lignes 16 à 17	Invocuez la fonction <i>rotation()</i> et affichez le résultat avec une rotation de 45 degrés.
Lignes 20 – 21	Invocuez la fonction <i>crop()</i> et transmettez deux points pour recadrer l'image et l'afficher.

Vous trouverez un aperçu des résultats dans la figure suivante :

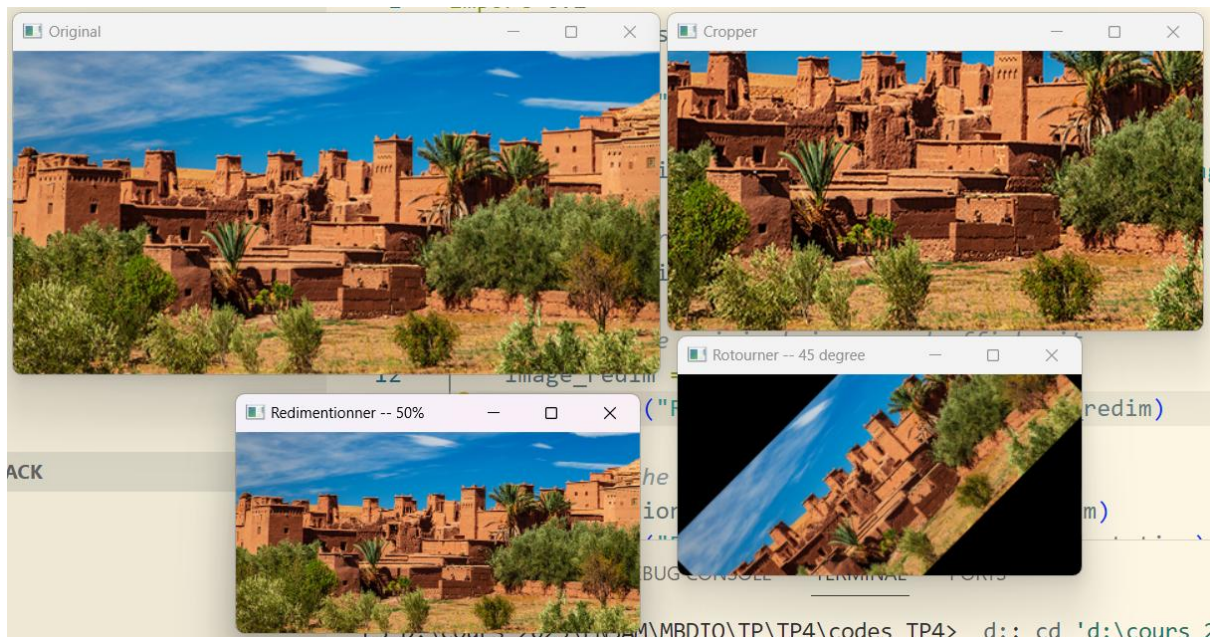


Figure 1: Image redimensionnée, croppée, retournée avec un angle de 45°

2. Fusion (superposer) des images

La fusion d'images fait référence au processus de combinaison de deux images pour créer une seule image présentant les caractéristiques des deux images originales. Le résultat est une combinaison des valeurs de pixels correspondantes des deux images originales avec des poids.

Le processus de fusion consiste à spécifier un poids pour chaque pixel de l'une des images originales, à calculer $(1 - \text{poids})$ pour chaque pixel d'une autre image, puis à les fusionner pour produire l'image de sortie.

La fusion d'images est une opération courante en vision par ordinateur et en traitement d'images et elle est utilisée pour diverses applications, telles que la création de panoramas, la composition d'images et la génération d'effets spéciaux dans des vidéos et des images.

Disons qu'il y a deux images, Image1 et Image2, comme le montrent les figures suivantes.



Figure 2: image 1



Figure 3: image 2

Le mélange d'images selon la documentation d'OpenCV obéit à la formule suivante :

$$g(i,j) = \alpha f_1(i,j) + (1 - \alpha) f_2(i,j)$$

$f_1(i,j)$ et $f_2(i,j)$ sont les deux images originales, et $g(i,j)$ est l'image de sortie. α est le poids (valeur de 0 à 1) sur la première image, $(1 - \alpha)$ est le poids sur la deuxième image.

En ajustant la valeur de α , nous pouvons obtenir différents effets de mélange. Comme dans les sections précédentes, la fonction `cv2.addWeighted()` est utilisée pour mélanger les deux images.

La fonction fusion sera ajoutée à la classe *TraitementImage* :

```
1 def fusion(self,fusion,alpha,image=None ):
2     if image is None:
3         image=self.image
4         fusion=cv2.resize(fusion,(image.shape[1],image.shape[0]))
5         resultat=cv2.addWeighted(image,alpha,fusion,(1.0-alpha),0)
6         return resultat
```

Explication :

Ligne 1	Définissez la fonction <i>fusion()</i> , les paramètres sont l'image à mélanger et l'alpha qui est compris entre 0 et 1.
Ligne 4	Les deux images doivent être de la même taille, rendez-les identiques en utilisant <i>cv2.resize()</i> .

3. Déformation d'image

La déformation d'image (Image Warp) fait référence au processus de transformation géométrique d'une image sous différentes formes. Cela implique d'appliquer une perspective ou une transformation affine à l'image, qui peut changer sa taille, son orientation et sa forme.

Le Warp est souvent utilisé pour corriger la distorsion provoquée par la perspective de la caméra lors de la capture d'une image. Cette distorsion fait apparaître les objets de l'image en taille et en forme différentes en fonction de leur position dans l'image. Par exemple, les objets plus proches de la caméra semblent plus grands et plus déformés que ceux plus éloignés.

Le processus de déformation de perspective consiste à définir quatre points dans l'image d'origine et à les mapper sur quatre points correspondants dans l'image de sortie. Ces points sont respectivement appelés points sources et points de destination. Une fois les points correspondants identifiés, une matrice de transformation est calculée, qui mappe chaque pixel de l'image d'origine à son nouvel emplacement dans l'image de sortie.

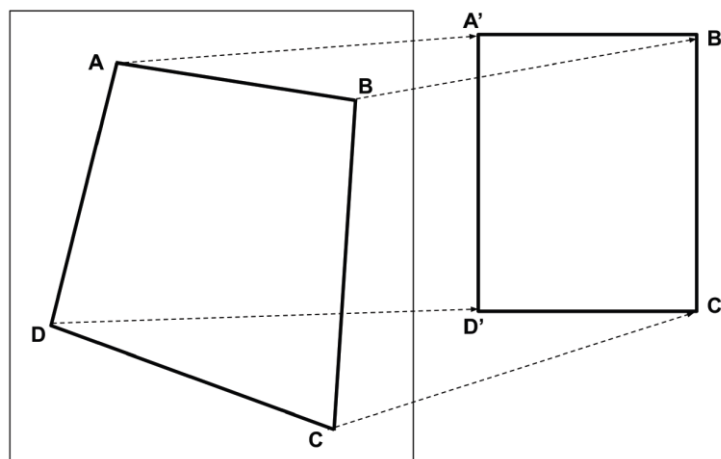


Figure 4. Déformation d'image

Dans cette partie de TP, à titre d'exemple dans la figure ci-dessous, l'image de gauche est prise par une caméra de tablette depuis la page d'accueil d'OpenCV.org, l'image semble

déformée, la déformation de perspective sera utilisée pour la corriger et l'aligner correctement, le résultat sera affiché sur le côté droit de la figure